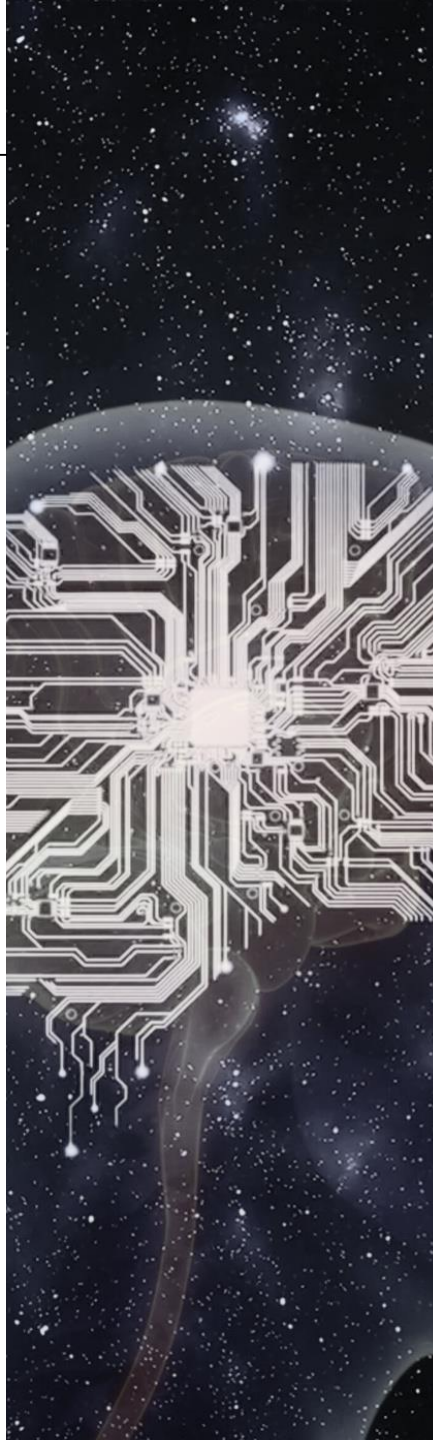
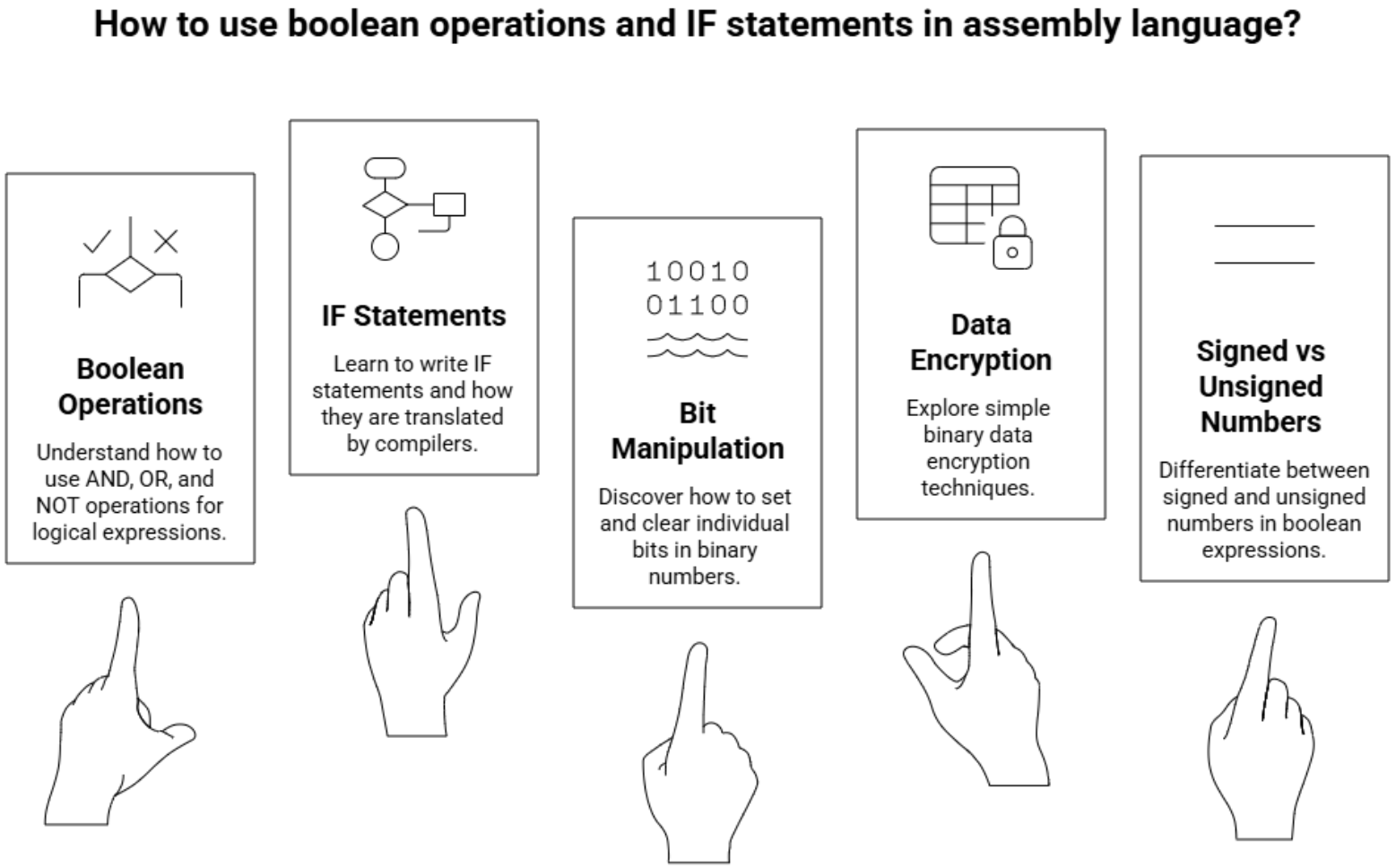
Assembly Programming

6. Conditional Processing



Assembly language, as primitive as it is, provides all the tools you need for decision-making logic.



Conditional Branching

TABLE 6-1 Selected Boolean Instructions.

Operation	Description
AND	Boolean AND operation between a source operand and a destination operand.
OR	Boolean OR operation between a source operand and a destination operand.
XOR	Boolean exclusive-OR operation between a source operand and a destination operand.
NOT	Boolean NOT operation on a destination operand.
TEST	Implied boolean AND operation between a source and destination operand, setting the CPU flags appropriately.

The CPU Status Flags

Boolean Instruction Flags

Flag	Zero	Carry	Sign	Overflow	Parity
Description	Result of operation equals zero	Operation generates carry out	Copy of high bit of operand	Result outside signed range	Even number of 1 bits

Made with Napkin

Boolean and Comparison Instructions

AND Instruction

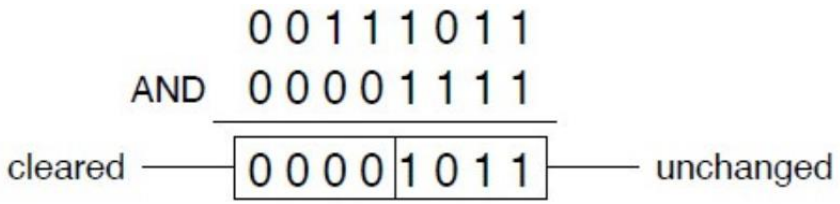
bit mask

The AND instruction lets you clear 1 or more bits in an operand without affecting other bits.

```
AND  destination,source

AND reg,reg
AND reg,mem
AND reg,imm
AND mem,reg
AND mem,imm
```

x	y	x ∧ y
0	0	0
0	1	0
1	0	0
1	1	1




```
mov al,10101110b
and al,11110110b              ; result in AL = 10100110
```

The AND instruction always clears the Overflow and Carry flags.
It modifies the Sign, Zero, and Parity flags.

Converting Characters to Upper case

The AND instruction provides an easy way to translate a letter from lowercase to uppercase.

```
0 1 1 0 0 0 0 1 = 61h ('a')
0 1 0 0 0 0 0 1 = 41h ('A')
```

```
.data
array BYTE 50 DUP(?)

.code
        mov ecx,LENGTHOF array
        mov esi,OFFSET array
L1:     and BYTE PTR [esi],11011111b        ; clear bit 5
        inc esi
        loop L1
```

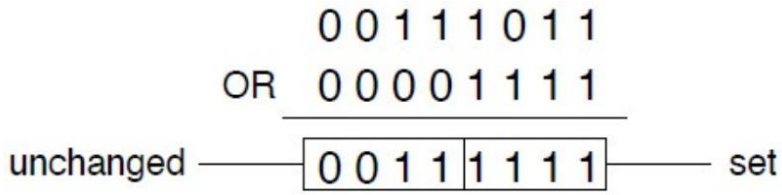
OR Instruction

The OR instruction performs a boolean OR operation between each pair of matching bits in two operands and places the result in the destination operand:

```
OR  destination,source

OR reg,reg
OR reg,mem
OR reg,imm
OR mem,reg
OR mem,imm
```

x	y	x ∨ y
0	0	0
0	1	1
1	0	1
1	1	1




```
mov al,11100011b
or  al,00000100b              ; result in AL = 11100111
```

The OR instruction always clears the Carry and Overflow flags.
It modifies the Sign, Zero, and Parity flags

Bit-Mapped Sets

array elements 0, 1, 2, and 31 are members of the set named SetX:

```
SetX = 10000000 00000000 00000000 00000111

mov  eax,SetX
and  eax,10000b                    ; is element[4] a member of SetX?
```

If the AND instruction in this example clears the Zero flag, we know that element [4] is a member of SetX.

Set Complement

The complement of a set can be generated using the NOT instruction, which reverses all bits.

```
mov  eax,SetX
not  eax                           ; complement of SetX
```

Set Intersection

The AND instruction produces a bit vector that represents the intersection of two sets.

```
mov  eax,SetX
and  eax,SetY

        10000000000000000000000000000111    (SetX)
AND     10000010101000000000011101100011    (SetY)
-------------------------------------------------------
        10000000000000000000000000000011    (intersection)
```

Set Union

The OR instruction produces a bit map that represents the union of two sets.

```
mov  eax,SetX
or   eax,SetY

        10000000000000000000000000000111    (SetX)
OR      10000010101000000000011101100011    (SetY)
-------------------------------------------------------
        10000010101000000000011101100111    (union)
```

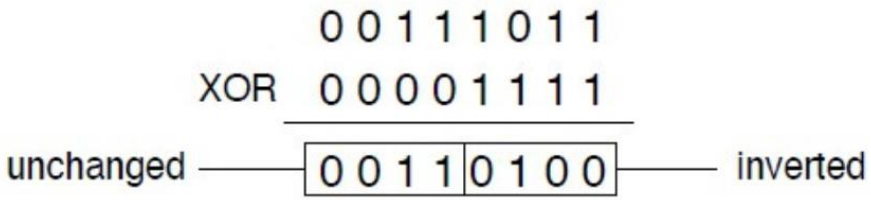
XOR Instruction

The XOR instruction performs a boolean exclusive-OR operation between each pair of matching bits in two operands and stores the result in the destination operand

```
XOR  destination,source
```

x	y	x ⊕ y
0	0	0
0	1	1
1	0	1
1	1	0

x	y	x ⊕ y	(x ⊕ y) ⊕ y
0	0	0	0
0	1	1	0
1	0	1	1
1	1	0	1



The XOR instruction always clears the Overflow and Carry flags.
XOR modifies the Sign, Zero, and Parity flags

Checking the Parity Flag

An effective way to check the parity of a number without changing its value is to exclusive-OR the number with zero:

```
mov  al,10110101b          ; 5 bits = odd parity
xor  al,0                  ; Parity flag clear (odd)
mov  al,11001100b          ; 4 bits = even parity
xor  al,0                  ; Parity flag set (even)


mov  ax,64C1h              ; 0110 0100 1100 0001
xor  ah,al                 ; Parity flag set (even)
```

NOT Instruction

The NOT instruction toggles (inverts) all bits in an operand.
The result is called the one's complement.

```
NOT reg
NOT mem
```

```
mov  al,11110000b
not  al                          ; AL = 00001111b
```

```
NOT   00111011
      --------
      11000100 ---- inverted
```

No flags are affected by the NOT instruction.

TEST Instruction

The TEST instruction performs an implied AND operation between each pair of matching bits in two operands and sets the Sign, Zero, and Parity flags based on the value assigned to the destination operand.
The only difference between TEST and AND is that TEST does not modify the destination operand.

Example: Testing Multiple Bits

```
0 0 1 0 0 1 0 1  <- input value
0 0 0 0 1 0 0 1  <- test value
0 0 0 0 0 0 0 1  <- result: ZF = 0

0 0 1 0 0 1 0 0  <- input value
0 0 0 0 1 0 0 1  <- test value
0 0 0 0 0 0 0 0  <- result: ZF = 1
```

The TEST instruction always clears the Overflow and Carry flags.
It modifies the Sign, Zero, and Parity flags in the same way as the AND instruction.

CMP Instruction

The CMP (compare) instruction performs an implied subtraction of a source operand from a destination operand. Neither operand is modified

```
CMP destination,source
```

```
if A > B ...
while X > 0 and X < 200  ...
if check_for_error( N ) = true
```

The CMP instruction changes the Overflow, Sign, Zero, Carry, Auxiliary Carry, and Parity flags according to the value the destination operand would have had if actual subtraction had taken place.
When two unsigned operands are compared, the Zero and Carry flags indicate the following relations between operands:

CMP Results	ZF	CF
Destination < source	0	1
Destination > source	0	0
Destination = source	1	0

unsigned operands are compared

CMP Results	Flags
Destination < source	SF ≠ OF
Destination > source	SF = OF
Destination = source	ZF = 1

signed operands are compared

Boolean and Comparison Instructions

Examples

```
mov  ax,5
cmp  ax,10                    ; ZF = 0 and CF = 1


mov  ax,1000
mov  cx,1000
cmp  cx,ax                    ; ZF = 1 and CF = 0


mov  si,105
cmp  si,0                     ; ZF = 0 and CF = 0
```

Setting and Clearing Individual CPU Flags

```
test  al,0                    ; set Zero flag
and   al,0                    ; set Zero flag
or    al,1                    ; clear Zero flag

or   al,80h                   ; set Sign flag
and  al,7Fh                   ; clear Sign flag

stc                           ; set Carry flag
clc                           ; clear Carry flag

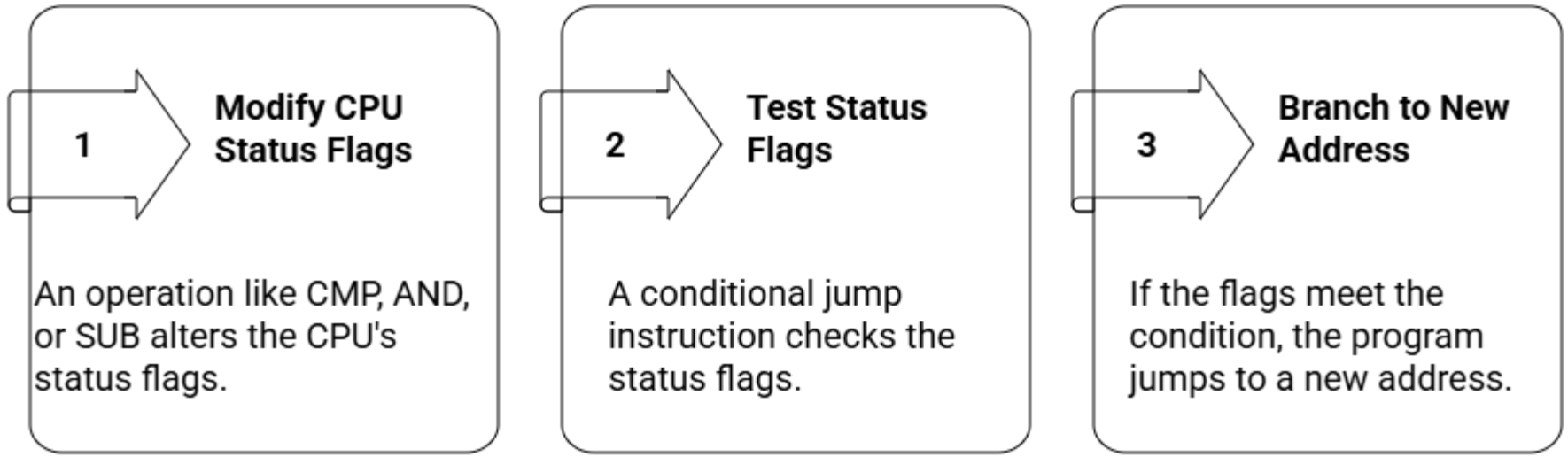
mov  al,7Fh                   ; AL = +127
inc  al                       ; AL = 80h (-128), OF=1
or   eax,0                    ; clear Overflow flag
```

Boolean and Comparison Instructions

Conditional Structures

There are no explicit high-level logic structures in the x86 instruction set, but you can implement them using a combination of comparisons and jumps.

Conditional Statement Execution in x86



Made with Napkin

Conditional Jumps

Example 1

```
        cmp   eax,0
        jz    L1                    ; jump if ZF = 1
        .
        .
L1:
```

Example 2

```
        and   dl,10110000b
        jnz   L2                    ; jump if ZF = 0
        .
        .
L2:
```

Jcond Instruction

A conditional jump instruction branches to a destination label when a status flag condition is true.

```
Jcond destination
```

JC	Jump if carry (Carry flag set)
JNC	Jump if not carry (Carry flag clear)
JZ	Jump if zero (Zero flag set)
JNZ	Jump if not zero (Zero flag clear)

Conditional Jumps

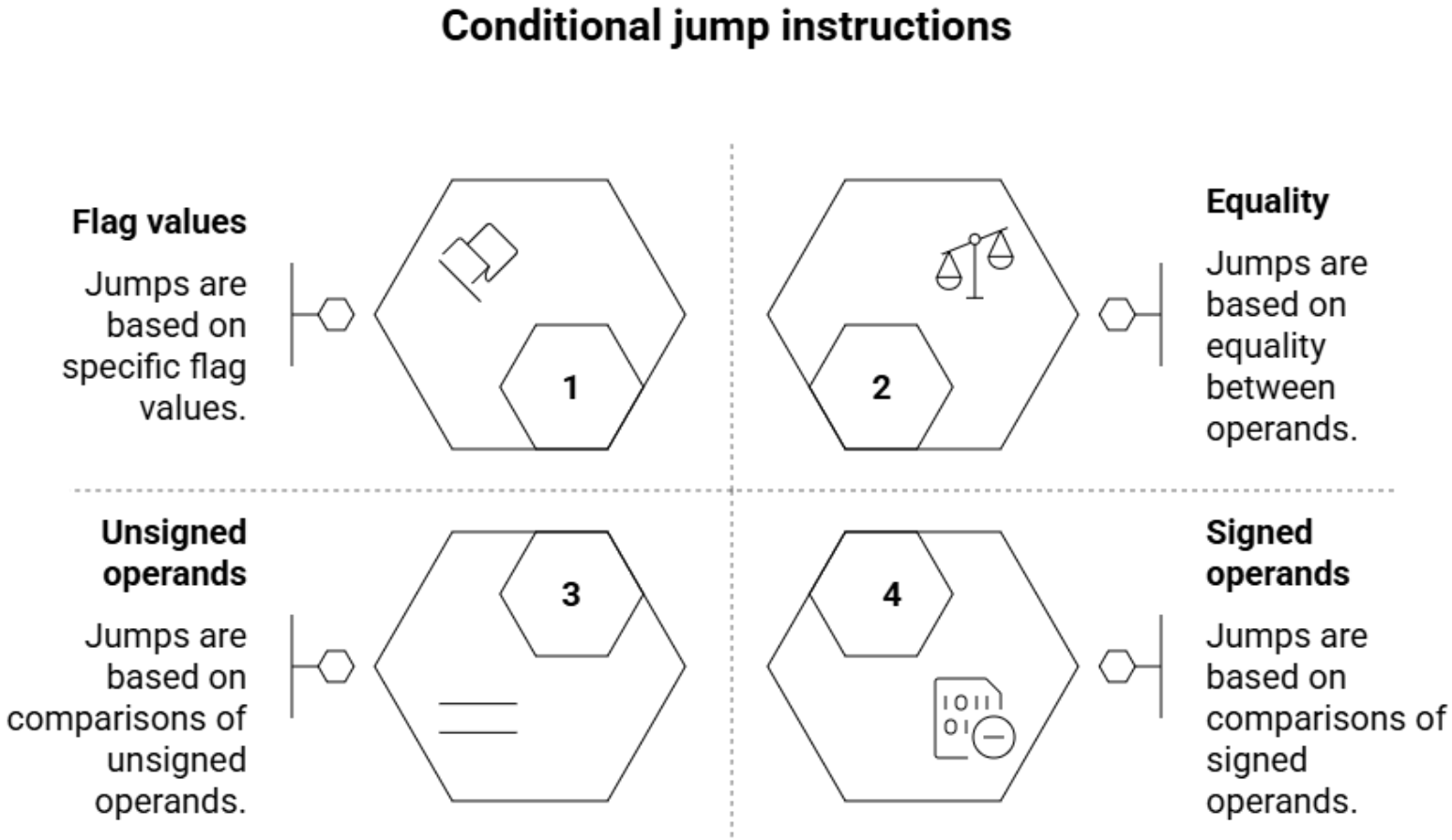
Using the CMP Instruction

```
cmp   eax,5
je    L1                        ; jump if equal

mov   ax,5
cmp   ax,6
jl    L1                        ; jump if less

mov   ax,5
cmp   ax,4
jg    L1                        ; jump if greater
```

Types of Conditional Jump Instructions



Made with Napkin

Table 6-2 Jumps Based on Specific Flag Values.

Mnemonic	Description	Flags / Registers
JZ	Jump if zero	ZF = 1
JNZ	Jump if not zero	ZF = 0
JC	Jump if carry	CF = 1
JNC	Jump if not carry	CF = 0
JO	Jump if overflow	OF = 1
JNO	Jump if not overflow	OF = 0
JS	Jump if signed	SF = 1
JNS	Jump if not signed	SF = 0
JP	Jump if parity (even)	PF = 1
JNP	Jump if not parity (odd)	PF = 0

Equality Comparisons

Table 6-3 Jumps Based on Equality.

Mnemonic	Description
JE	Jump if equal (*leftOp* = *rightOp*)
JNE	Jump if not equal (*leftOp* ≠ *rightOp*)
JCXZ	Jump if CX = 0
JECXZ	Jump if ECX = 0
JRCXZ	Jump if RCX = 0 (64-bit mode)

Conditional Jumps

Example 1:

```
mov  edx,0A523h
cmp  edx,0A523h
jne  L5                          ; jump not taken
je   L1                          ; jump is taken
```

Example 2:

```
mov  bx,1234h
sub  bx,1234h
jne  L5                          ; jump not taken
je   L1                          ; jump is taken
```

Example 3:

```
mov    cx,0FFFFh
inc    cx
jcxz   L2                        ; jump is taken
```

Example 4:

```
xor    ecx,ecx
jecxz  L2                        ; jump is taken
```

Unsigned Comparisons

Table 6-4 Jumps Based on Unsigned Comparisons.

Mnemonic	Description
JA	Jump if above (if *leftOp* > *rightOp*)
JNBE	Jump if not below or equal (same as JA)
JAE	Jump if above or equal (if *leftOp* ≥ *rightOp*)
JNB	Jump if not below (same as JAE)
JB	Jump if below (if *leftOp* < *rightOp*)
JNAE	Jump if not above or equal (same as JB)
JBE	Jump if below or equal (if *leftOp* ≤ *rightOp*)
JNA	Jump if not above (same as JBE)

Signed Comparisons

Table 6-5 Jumps Based on Signed Comparisons.

Mnemonic	Description
JG	Jump if greater (if *leftOp* > *rightOp*)
JNLE	Jump if not less than or equal (same as JG)
JGE	Jump if greater than or equal (if *leftOp* ≥ *rightOp*)
JNL	Jump if not less (same as JGE)
JL	Jump if less (if *leftOp* < *rightOp*)
JNGE	Jump if not greater than or equal (same as JL)
JLE	Jump if less than or equal (if *leftOp* ≤ *rightOp*)
JNG	Jump if not greater (same as JLE)

Conditional Jumps

Example 1

```
mov    edx,-1
cmp    edx,0
jnl    L5                    ; jump not taken (-1 >= 0 is false)
jnle   L5                    ; jump not taken (-1 > 0 is false)
jl     L1                    ; jump is taken (-1 < 0 is true)
```

Example 2

```
mov    bx,+32
cmp    bx,-35
jng    L5                    ; jump not taken (+32 <= -35 is false)
jnge   L5                    ; jump not taken (+32 < -35 is false)
jge    L1                    ; jump is taken (+32 >= -35 is true)
```

Example 3

```
mov  ecx,0
cmp  ecx,0
jg   L5                      ; jump not taken (0 > 0 is false)
jnl  L1                      ; jump is taken (0 >= 0 is true)
```

Example 4

```
mov  ecx,0
cmp  ecx,0
jl   L5                      ; jump not taken (0 < 0 is false)
jng  L1                      ; jump is taken (0 <= 0 is true)
```

Conditional Jump Applications

Testing Status Bits

```
mov   al,status
test  al,00100000b              ; test bit 5
jnz   DeviceOffline

mov   al,status
test  al,00010011b              ; test bits 0,1,4
jnz   InputDataByte

mov   al,status
and   al,10001100b              ; mask bits 2,3,7
cmp   al,10001100b              ; all bits set?
je    ResetMachine              ; yes: jump to label
```

Larger of Two Integers

```
    mov  edx,eax          ; assume EAX is larger
    cmp  eax,ebx          ; if EAX is >= EBX
    jae  L1               ;    jump to L1
    mov  edx,ebx          ; else move EBX to EDX
L1:                       ; EDX contains the larger integer
```

Smallest of Three Integers

```
.data
V1 WORD ?
V2 WORD ?
V3 WORD ?
.code
    mov   ax,V1           ; assume V1 is smallest
    cmp   ax,V2           ; if AX <= V2
    jbe   L1              ;   jump to L1
    mov   ax,V2           ; else move V2 to AX
L1: cmp   ax,V3           ; if AX <= V3
    jbe   L2              ;   jump to L2
    mov   ax,V3           ; else move V3 to AX
L2:
```

Conditional Jumps

Loop until Key Pressed

```
.data
char BYTE ?
.code
L1:  mov  eax,10              ; create 10 ms delay
     call Delay
     call ReadKey             ; check for key
     jz   L1                  ; repeat if no key
     mov  char,AL             ; save the character
```

Application: Sequential Search of an Array

```
; Scanning an Array              (ArrayScan.asm)
; Scan an array for the first nonzero value.
INCLUDE Irvine32.inc
.data
intArray  SWORD  0,0,0,0,1,20,35,-12,66,4,0
;intArray SWORD  1,0,0,0                    ; alternate test data
;intArray SWORD  0,0,0,0                    ; alternate test data
;intArray SWORD  0,0,0,1                    ; alternate test data
noneMsg  BYTE "A non-zero value was not found",0

.code
main PROC
        mov ebx,OFFSET intArray         ; point to the array
        mov ecx,LENGTHOF intArray       ; loop counter
L1:     cmp   WORD PTR [ebx],0          ; compare value to zero
        jnz   found                     ; found a value
        add   ebx,2                     ; point to next
        loop  L1                        ; continue the loop
        jmp   notFound                  ; none found
```

Conditional Jumps

```
found:                                          ; display the value
        movsx eax,WORD PTR[ebx]                 ; sign-extend into EAX
        call  WriteInt
        jmp   quit
notFound:                                       ; display "not found" message
        mov   edx,OFFSET noneMsg
        call  WriteString
quit:
        call Crlf
        exit
main ENDP
END main
```

Application: Simple String Encryption

```
; Encryption Program                    (Encrypt.asm)

INCLUDE Irvine32.inc
KEY = 239                                       ; any value between 1-255
BUFMAX = 128                                    ; maximum buffer size

.data
sPrompt  BYTE "Enter the plain text:",0
sEncrypt BYTE "Cipher text:",0
sDecrypt BYTE "Decrypted:",0
buffer   BYTE BUFMAX+1 DUP(0)
bufSize  DWORD ?
.code
main PROC
        call InputTheString             ; input the plain text
        call TranslateBuffer            ; encrypt the buffer
        mov edx,OFFSET sEncrypt         ; display encrypted message
        call DisplayMessage
        call TranslateBuffer            ; decrypt the buffer
        mov edx,OFFSET sDecrypt         ; display decrypted message
        call DisplayMessage
exit
main ENDP
```

Conditional Jumps

```
;-----------------------------------------------------
InputTheString PROC
;
; Prompts user for a plaintext string. Saves the string
; and its length.
; Receives: nothing
; Returns: nothing
;-----------------------------------------------------
        pushad                          ; save 32-bit registers
        mov edx,OFFSET sPrompt          ; display a prompt
        call WriteString
        mov ecx,BUFMAX                  ; maximum character count
        mov edx,OFFSET buffer           ; point to the buffer
        call ReadString                 ; input the string
        mov bufSize,eax                 ; save the length
        call Crlf
        popad
        ret
InputTheString ENDP
```

Conditional Jumps

```
;-----------------------------------------------------
DisplayMessage PROC
;
; Displays the encrypted or decrypted message.
; Receives: EDX points to the message
; Returns:  nothing
;-----------------------------------------------------
                pushad
                call WriteString
                mov edx,OFFSET buffer           ; display the buffer
                call WriteString
                call Crlf
                call Crlf
                popad
                ret
DisplayMessage ENDP
```

Conditional Jumps

```
;-----------------------------------------------
TranslateBuffer PROC
;
; Translates the string by exclusive-ORing each
; byte with the encryption key byte.
; Receives: nothing
; Returns: nothing
;-----------------------------------------------
        pushad
        mov ecx,bufSize             ; loop counter
        mov esi,0                   ; index 0 in buffer
L1:
        xor buffer[esi],KEY         ; translate a byte
        inc esi                     ; point to next byte
        loop L1
        popad
        ret
TranslateBuffer ENDP
END main
```

Conditional Jumps

Conditional Loop Instructions

LOOPZ and LOOPE Instructions

The LOOPZ (loop if zero) instruction works just like the LOOP instruction except that it has one additional condition: the Zero flag must be set in order for control to transfer to the destination label.

```
LOOPZ destination
```

```
ECX = ECX - 1
if ECX > 0 and ZF = 1, jump to destination
```

LOOPNZ and LOOPNE Instructions

The LOOPNZ (loop if not zero) instruction is the counterpart of LOOPZ. The loop continues while the unsigned value of ECX is greater than zero (after being decremented) and the Zero flag is clear.

```
LOOPNZ destination
```

```
ECX = ECX - 1
if ECX > 0 and ZF = 0, jump to destination
```

Conditional Loop Instructions

Example

```
.data
array    SWORD -3, -6, -1, -10, 10, 30, 40, 4
sentinel SWORD 0

.code
            mov esi, OFFSET array
            mov ecx, LENGTHOF array
next:
            test WORD PTR [esi], 8000h      ; test sign bit
             pushfd                         ; push flags on stack
            add esi, TYPE array
            popfd                           ; pop flags from stack
            loopnz next                     ; continue loop
            jnz quit                        ; none found
            sub esi, TYPE array             ; ESI points to value
quit:
```

```
.data
array SWORD 50 DUP(?)
sentinel SWORD 0FFFFh

.code
            mov esi, OFFSET array
            mov ecx, LENGTHOF array
L1:         cmp WORD PTR [esi], 0
            pushfd
            add esi, TYPE array
             popfd
            loope L1
            jz quit
            sub esi, TYPE array
quit:
```

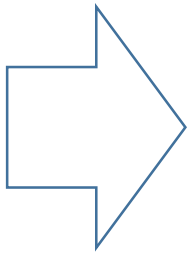
Conditional Loop Instructions

Conditional Structures

Block-Structured IF Statements

Example 1

```
if( op1 == op2 )
{
     X = 1;
     Y = 2;
}
```

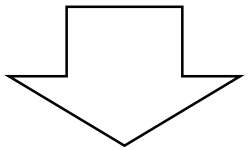


```
     mov   eax,op1
     cmp   eax,op2              ; op1 == op2?
     jne   L1                   ; no: skip next
     mov   X,1                  ; yes: assign X and Y
     mov   Y,2
L1:
```

```
     mov   eax,op1
     cmp   eax,op2              ; op1 == op2?
     je    L1                   ; yes: jump to L1
     jmp   L2                   ; no: skip assignments
L1:  mov   X,1                  ; assign X and Y
     mov   Y,2
L2:
```

Example 2

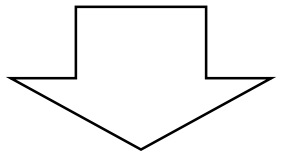
```
clusterSize = 8192;
if terrabytes < 16
  clusterSize = 4096;
```



```
     mov    clusterSize,8192     ; assume larger cluster
     cmp    terrabytes, 16       ; smaller than 16 TB?
     jae    next
     mov    clusterSize,4096     ; switch to smaller cluster
next:
```

Example 3

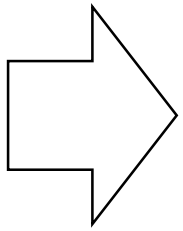
```
if op1 > op2
     call Routine1
else
     call Routine2
end if
```



```
      mov   eax,op1                    ; move op1 to a register
      cmp   eax,op2                    ; op1 > op2?
      jg    A1                         ; yes: call Routine1
      call  Routine2                   ; no: call Routine2
      jmp   A2                         ; exit the IF statement
A1:   call  Routine1
A2:
```

White Box Testing

```
if op1 == op2
  if X > Y
    call Routine1
  else
    call Routine2
  end if
else
  call Routine3
end if
```



```
1:        mov   eax,op1
2:        cmp   eax,op2                    ; op1 == op2?
3:        jne   L2                         ; no: call Routine3

; process the inner IF-ELSE statement.
4:        mov   eax,X
5:        cmp   eax,Y                      ; X > Y?
6:        jg    L1                         ; yes: call Routine1
7:        call  Routine2                   ; no: call Routine2
8:        jmp   L3                         ; and exit
9:   L1: call  Routine1                    ; call Routine1
10:       jmp   L3                         ; and exit
11:  L2: call  Routine3
12:  L3:
```

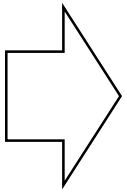
Table 6-6 Testing the Nested IF Statement.

op1	op2	X	Y	Line Execution Sequence	Calls
10	20	30	40	1, 2, 3, 11, 12	Routine3
10	20	40	30	1, 2, 3, 11, 12	Routine3
10	10	30	40	1, 2, 3, 4, 5, 6, 7, 8, 12	Routine2
10	10	40	30	1, 2, 3, 4, 5, 6, 9, 10, 12	Routine1

Compound Expressions

Logical AND Operator

```
if (a1 > b1) AND (b1 > c1)
    X = 1
end if
```

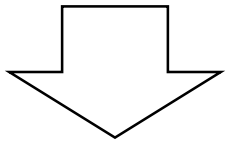


```
     cmp   a1,b1                ; first expression...
     ja    L1
     jmp   next
L1:  cmp   b1,c1                ; second expression...
     ja    L2
     jmp   next
L2:  mov   X,1                  ; both true: set X to 1
next:
```

```
     cmp   a1,b1                ; first expression...
     jbe   next                 ; quit if false
     cmp   b1,c1                ; second expression
     jbe   next                 ; quit if false
     mov   X,1                  ; both are true
next:
```

Logical OR Operator

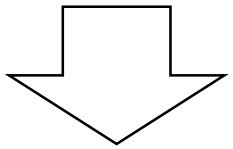
```
if (a1 > b1) OR (b1 > c1)
   X = 1
```



```
     cmp   a1,b1                ; 1: compare AL to BL
     ja    L1                   ; if true, skip second expression
     cmp   b1,c1                ; 2: compare BL to CL
     jbe   next                 ; false: skip next statement
L1:  mov   X,1                  ; true: set X = 1
next:
```

WHILE Loops

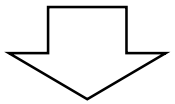
```
while( val1 < val2 )
{
     val1++;
     val2--;
}
```



```
     mov   eax,val1              ; copy variable to EAX
beginwhile:
     cmp   eax,val2              ; if not (val1 < val2)
     jnl   endwhile              ;   exit the loop
     inc   eax                   ; val1++;
     dec   val2                  ; val2--;
     jmp   beginwhile            ; repeat the loop
endwhile:
     mov   val1,eax              ; save new value for val1
```

Example: IF statement Nested in a Loop

```
int array[] = {10,60,20,33,72,89,45,65,72,18};
int sample = 50;
int ArraySize = sizeof array / sizeof sample;
int index = 0;
int sum = 0;
while( index < ArraySize )
{
        if( array[index] > sample )
        {
                sum += array[index];
        }
        index++;
}
```



```
.data
sum         DWORD 0
sample      DWORD 50
array       DWORD 10,60,20,33,72,89,45,65,72,18
ArraySize = ($ - Array) / TYPE array
.code
main PROC
            mov eax,0                   ; sum
            mov edx,sample
            mov esi,0                   ; index
            mov ecx,ArraySize
L1:         cmp esi,ecx                 ; if esi < ecx
            jl  L2
            jmp L5
L2:         cmp array[esi*4], edx       ; if array[esi] > edx
            jg  L3
            jmp L4
L3:         add eax,array[esi*4]
L4:         inc esi
            jmp L1
L5:         mov sum,eax
```

Conditional Structures

Figure 6–1 Loop containing IF statement.

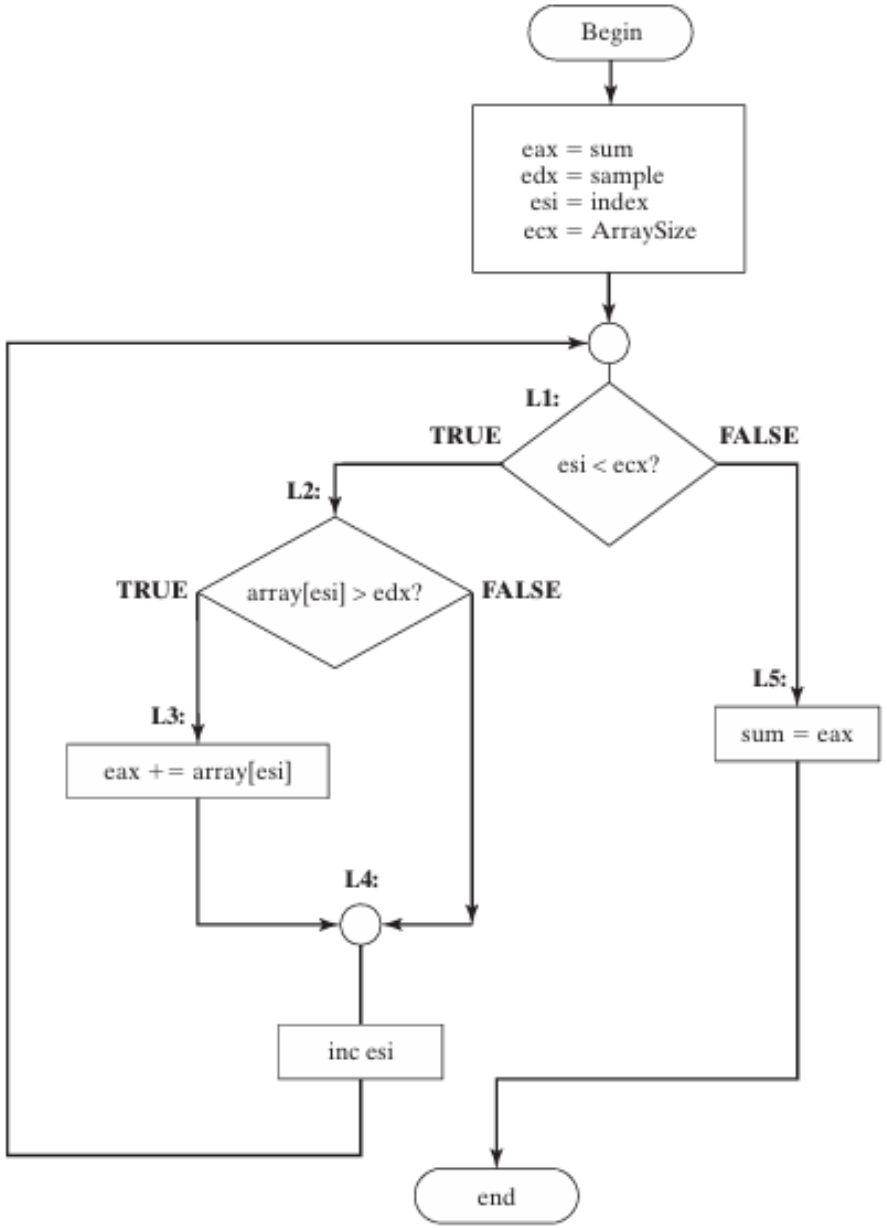
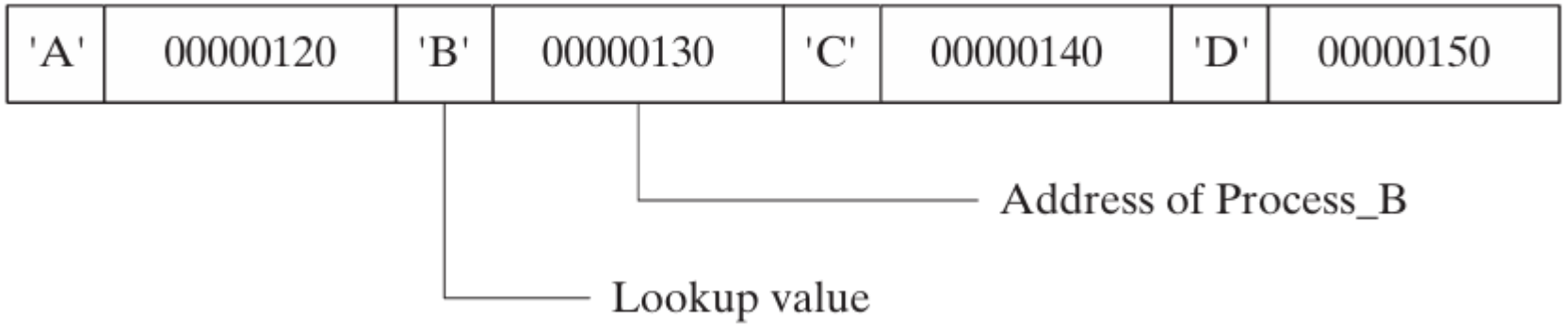


Table-Driven Selection

Table-driven selection is a way of using a table lookup to replace a multiway selection structure. To use it, you must create a table containing lookup values and the offsets of labels or procedures, and then you must use a loop to search the table.

```
.data
CaseTable BYTE   'A'              ; lookup value
    DWORD Process_A               ; address of procedure
    BYTE  'B'
    DWORD Process_B
     (etc.)
```

FIGURE 6–2 Table of procedure offsets.



Example Program

```
; Table of Procedure Offsets              (ProcTable.asm)
; This program contains a table with offsets of procedures.
; It uses the table to execute indirect procedure calls.
INCLUDE Irvine32.inc
.data
CaseTable BYTE 'A'                                  ; lookup value
          DWORD   Process_A                         ; address of procedure
EntrySize = ($ - CaseTable)
          BYTE 'B'
          DWORD   Process_B
          BYTE 'C'
          DWORD   Process_C
          BYTE 'D'
          DWORD   Process_D
NumberOfEntries = ($ - CaseTable) / EntrySize
prompt    BYTE "Press capital A,B,C,or D: ",0
msgA      BYTE "Process_A",0
msgB      BYTE "Process_B",0
msgC      BYTE "Process_C",0
msgD      BYTE "Process_D",0
```

```
.code
main PROC
        mov edx,OFFSET prompt       ; ask user for input
        call WriteString
        call ReadChar               ; read character into AL
        mov ebx,OFFSET CaseTable    ; point EBX to the table
        mov ecx,NumberOfEntries     ; loop counter
L1:
        cmp al,[ebx]                ; match found?
        jne L2                      ; no: continue
        call NEAR PTR [ebx + 1]     ; yes: call the procedure
        call WriteString            ; display message
        call Crlf
        jmp L3                      ; exit the search
L2:
        add ebx,EntrySize           ; point to the next entry
        loop L1                     ; repeat until ECX = 0
L3:
        exit
main ENDP
```

```
Process_A PROC
        mov edx,OFFSET msgA
        ret
Process_A ENDP

Process_B PROC
        mov edx,OFFSET msgB
        ret
Process_B ENDP

Process_C PROC
        mov edx,OFFSET msgC
        ret
Process_C ENDP

Process_D PROC
        mov edx,OFFSET msgD
        ret
Process_D ENDP
END main
```

Conditional Structures

Conditional Control Flow Directives

In 32-bit mode, MASM includes a number of high-level conditional control flow directives that help to simplify the coding of conditional statements. Unfortunately, they cannot be used in 64-bit mode.

Table 6-7 Conditional Control Flow Directives.

Directive	Description
.BREAK	Generates code to terminate a .WHILE or .REPEAT block
.CONTINUE	Generates code to jump to the top of a .WHILE or .REPEAT block
.ELSE	Begins block of statements to execute when the .IF condition is false
.ELSEIF *condition*	Generates code that tests *condition* and executes statements that follow, until an .ENDIF directive or another .ELSEIF directive is found
.ENDIF	Terminates a block of statements following an .IF, .ELSE, or .ELSEIF directive
.ENDW	Terminates a block of statements following a .WHILE directive
.IF *condition*	Generates code that executes the block of statements if *condition* is true.
.REPEAT	Generates code that repeats execution of the block of statements until *condition* becomes true
.UNTIL *condition*	Generates code that repeats the block of statements between .REPEAT and .UNTIL until *condition* becomes true
.UNTILCXZ	Generates code that repeats the block of statements between .REPEAT and .UNTILCXZ until *CX* equals zero
.WHILE *condition*	Generates code that executes the block of statements between .WHILE and .ENDW as long as *condition* is true

Creating IF Statements

The .IF, .ELSE, .ELSEIF, and .ENDIF directives make it easy for you to code multiway branching logic.

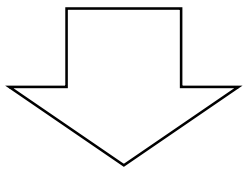
```
.IF condition1
    statements
[.ELSEIF condition2
    statements ]
[.ELSE
    statements ]
.ENDIF
```

Table 6-8 Runtime Relational and Logical Operators.

Operator	Description
expr1 == *expr2*	Returns true when *expr1* is equal to *expr2*.
expr1 != *expr2*	Returns true when *expr1* is not equal to *expr2*.
expr1 > *expr2*	Returns true when *expr1* is greater than *expr2*.
expr1 ⩾ *expr2*	Returns true when *expr1* is greater than or equal to *expr2*.
expr1 < *expr2*	Returns true when *expr1* is less than *expr2*.
expr1 ⩽ *expr2*	Returns true when *expr1* is less than or equal to *expr2*.
! *expr*	Returns true when *expr* is false.
expr1 && *expr2*	Performs logical AND between *expr1* and *expr2*.
expr1 ‖ *expr2*	Performs logical OR between *expr1* and *expr2*.
expr1 & *expr2*	Performs bitwise AND between *expr1* and *expr2*.
CARRY?	Returns true if the Carry flag is set.
OVERFLOW?	Returns true if the Overflow flag is set.
PARITY?	Returns true if the Parity flag is set.
SIGN?	Returns true if the Sign flag is set.
ZERO?	Returns true if the Zero flag is set.

Generating ASM Code

```
mov eax,6
.IF eax > val1
  mov result,1
.ENDIF
```



```
     mov   eax,6
     cmp   eax,val1
     jbe   @C0001                    ; jump on unsigned comparison
     mov   result,1
@C0001:
```

Signed and Unsigned Comparisons

```
.data
val1        DWORD   5
result      DWORD   ?
.code
            mov eax,6
            .IF eax > val1
                        mov result,1
            .ENDIF
```

```
            mov eax,6
            cmp eax,val1
            jbe @C0001                      ; jump on unsigned comparison
            mov result,1
@C0001:
```

Comparing a Signed Integer

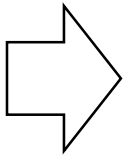
```
.data
val2 SDWORD -1
result DWORD ?
.code
     mov eax,6
     .IF eax > val2
       mov result,1
     .ENDIF
```

```
     mov eax,6
     cmp eax,val2
     jle @C0001                     ; jump on signed comparison
     mov result,1
@C0001:
```

Conditional Control Flow Directives

Comparing Registers

```
mov eax,6
mov ebx,val2
.IF eax > ebx
   mov result,1
.ENDIF
```



```
     mov  eax,6
     mov  ebx,val2
     cmp  eax, ebx
     jbe  @C0001
     mov  result,1
@C0001:
```

Compound Expressions

```
.IF expression1 || expression2
    statements
.ENDIF
```

```
.IF expression1 && expression2
    statements
.ENDIF
```

Creating Loops with .REPEAT and .WHILE

```
.REPEAT
    statements
.UNTIL condition
```

```
.WHILE condition
    statements
.ENDW
```

```
mov eax,0
.REPEAT
    inc  eax
    call WriteDec
    call Crlf
.UNTIL eax == 10
```

```
mov eax,0
.WHILE eax < 10
    inc  eax
    call WriteDec
    call Crlf
.ENDW
```

Example: Loop Containing an IF Statement

```
while( op1 < op2 )
{
     op1++;
     if( op1 == op3 )
       X = 2;
     else
       X = 3;
}
```

```
.data
X   DWORD 0
op1 DWORD 2                          ; test data
op2 DWORD 4                          ; test data
op3 DWORD 5                          ; test data
.code
     mov eax,op1
     mov ebx,op2
     mov ecx,op3
     .WHILE eax < ebx
       inc eax
       .IF eax == ecx
          mov X,2
       .ELSE
          mov X,3
       .ENDIF
     .ENDW
```

Conditional Control Flow Directives

Conditional Processing

6.1 Conditional Branching

6.2 Boolean and Comparison Instructions
- 6.2.1 The CPU Status Flags
- 6.2.2 AND Instruction
- 6.2.3 OR Instruction
- 6.2.4 Bit-Mapped Sets
- 6.2.5 XOR Instruction
- 6.2.6 NOT Instruction
- 6.2.7 TEST Instruction
- 6.2.8 CMP Instruction
- 6.2.9 Setting and Clearing Individual CPU Flags
- 6.2.10 Boolean Instructions in 64-Bit Mode
- 6.2.11 Section Review

6.3 Conditional Jumps
- 6.3.1 Conditional Structures
- 6.3.2 J*cond* Instruction
- 6.3.3 Types of Conditional Jump Instructions
- 6.3.4 Conditional Jump Applications
- 6.3.5 Section Review

6.4 Conditional Loop Instructions
- 6.4.1 LOOPZ and LOOPE Instructions
- 6.4.2 LOOPNZ and LOOPNE Instructions
- 6.4.3 Section Review

6.5 Conditional Structures
- 6.5.1 Block-Structured IF Statements
- 6.5.2 Compound Expressions
- 6.5.3 WHILE Loops
- 6.5.4 Table-Driven Selection
- 6.5.5 Section Review

6.6 Application: Finite-State Machines
- 6.6.1 Validating an Input String
- 6.6.2 Validating a Signed Integer
- 6.6.3 Section Review

6.7 Conditional Control Flow Directives
- 6.7.1 Creating IF Statements
- 6.7.2 Signed and Unsigned Comparisons
- 6.7.3 Compound Expressions
- 6.7.4 Creating Loops with .REPEAT and .WHILE

6.8 Chapter Summary

6.9 Key Terms
- 6.9.1 Terms
- 6.9.2 Instructions, Operators, and Directives

6.10 Review Questions and Exercises
- 6.10.1 Short Answer
- 6.10.2 Algorithm Workbench

6.11 Programming Exercises
- 6.11.1 Suggestions for Testing Your Code
- 6.11.2 Exercise Descriptions

reference

https://www.asmirvine.com/
https://intelligentcm.tistory.com/category/Assembly

Review questions and Exercises

&

Programming exercises

Github upload